ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 8

Базовые методы аутентификации

Рассмотрим процесс работы с JWT-токенами (JSON Web Token), научимся их создавать и валидировать. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

JSON Web Token

JSON Web Token (JWT) — открытый стандарт для создания токенов доступа, основанный на формате RFC 7519. Как правило, используется для передачи данных после аутентификации в клиент-серверных приложениях. Токены создаются сервером, подписываются секретным ключом и передаются клиенту, который в дальнейшем использует данный токен для подтверждения подлинности аккаунта.

В общем виде алгоритм работы JWT можно описать следующим образом:

1. Клиент выполняет вход, используя учетные данные, отправляя запрос на сервер.
2. Сервер проверяет эти учетные данные. Если они действительны, сервер генерирует JWT и отправляет его обратно клиенту.
3. Клиент сохраняет JWT, обычно в локальном хранилище, и включает его в заголовок каждого последующего HTTP-запроса.
4. Сервер, получая эти запросы, проверяет JWT. Если он действителен, клиент аутентифицирован и авторизован.

Как работает JWT

Токен JWT состоит из трёх частей: заголовка (header), полезной нагрузки (payload) и подписи или данных шифрования. Первые два элемента — это JSON-объекты определённой структуры. Третий элемент вычисляется на основании первых и зависит от выбранного алгоритма (в случае использования неподписанного JWT может быть

опущен). Токены могут быть перекодированы в компактное представление (JWS/JWE Compact Serialization): к заголовку и полезной нагрузке применяется алгоритм кодирования Base64-URL, после чего добавляется подпись и все три элемента разделяются точками («.»).

К примеру, для заголовка и полезной нагрузки, которые выглядят следующим образом:

```
{
  "alg": "HS512",
  "typ": "JWT"
}
{
  "sub": "12345",
  "name": "John Gold",
  "admin": true
}
```

получим следующее компактное представление (переносы строки добавлены для наглядности):


```
eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NSIsIm5hbWUiOiJKb2huIEdvbGQiLCJhZG1pbiI6dHJ1ZX0K.
LIHjWCBORSWMEibq-tnT8ue_deUqZx1K0XxCOXZRrBI
```


Заголовок

В заголовке указывается необходимая информация для описания самого токена.

Обязательный ключ здесь только один:

- **alg:** алгоритм, используемый для подписи или шифрования (в случае неподписанного JWT используется значение **«none»**).

Необязательные ключи:

- **typ:** тип токена (*type*). Используется в случае, когда токены смешиваются с другими объектами, имеющими JOSE заголовки. Должно иметь значение **«JWT»**.
- **cty:** тип содержимого (*content type*). Если в токене помимо зарегистрированных служебных ключей есть пользовательские, то данный ключ не должен присутствовать. В противном случае должно иметь значение **«JWT»**

Полезная нагрузка

В данной секции указывается пользовательская информация (например, имя пользователя и уровень его доступа), а также могут быть использованы некоторые служебные ключи. Все они являются необязательными:

- **iss:** чувствительная к регистру строка или URI, которая является уникальным идентификатором стороны, генерирующей токен (*issuer*).
- **sub:** чувствительная к регистру строка или URI, которая является уникальным идентификатором стороны, о которой содержится информация в данном токене (*subject*). Значения с этим ключом должны быть уникальны в контексте стороны, генерирующей JWT.
- **aud:** массив чувствительных к регистру строк или URI, являющийся списком получателей данного токена. Когда принимающая сторона получает JWT с данным ключом, она должна проверить наличие себя в получателях — иначе проигнорировать токен (*audience*).
- **exp:** время в формате Unix Time, определяющее момент, когда токен станет невалидным (*expiration*).
- **nbf:** в противоположность ключу **exp,** это время в формате Unix Time, определяющее момент, когда токен станет валидным (*not before*).
- **jti:** строка, определяющая уникальный идентификатор данного токена (*JWT ID*).
- **iat:** время в формате Unix Time, определяющее момент, когда токен был создан. **iat** и **nbf** могут не совпадать, например, если токен был создан раньше, чем время, когда он должен стать валидным (*issued at*).

Access- и refresh-токены

- Access-токен — это токен, который предоставляет доступ его владельцу к защищённым ресурсам сервера. Обычно он имеет короткий срок жизни и может нести в себе дополнительную информацию, такую как IP-адрес стороны, запрашивающей данный токен.
- Refresh-токен — это токен, позволяющий клиентам запрашивать новые access-токены по истечении их времени жизни. Данные токены обычно выдаются на длительный срок.

Схема работы

Если более подробно рассматривать процесс аутентификации в клиент-серверных приложениях с использованием JWT токенов, то, как правило, он реализован следующим образом:

1. Клиент проходит аутентификацию в приложении (к примеру, с использованием логина и пароля).
2. В случае успешной аутентификации сервер отправляет клиенту access- и refresh-токены.
3. При дальнейшем обращении к серверу клиент использует access-токен. Сервер проверяет токен на валидность и предоставляет клиенту доступ к ресурсам.
4. В случае, если access-токен становится не валидным, клиент отправляет refresh-токен, в ответ на который сервер предоставляет два обновлённых токена.

5. В случае, если refresh-токен становится не валидным, клиент опять должен пройти процесс аутентификации

Пример работы в node.js

Для работы с JWT токенами в node.js, необходимо установить библиотеку `jsonwebtoken`:

```
npm i jsonwebtoken
```

Простой пример кода для реализации маршрутов входа и регистрации будет выглядеть следующим образом:

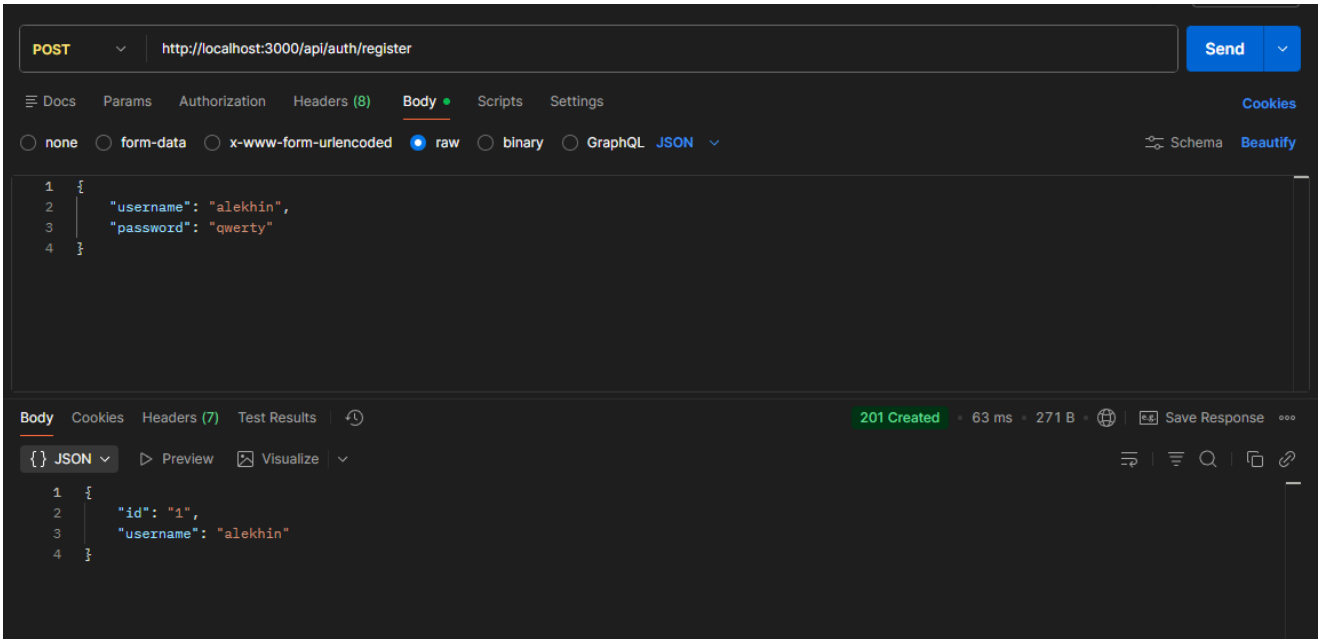
```
const express = require("express");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");

const app = express();
app.use(express.json());

const PORT = 3000;
// Рандомная строка, задающая подпись (гарантия, что токен выдан именно
твоим сервером)
const JWT_SECRET = "access_secret";
// Время жизни токена
const ACCESS_EXPIRES_IN = "15m";

// { id, username, passwordHash }
const users = [];

app.post("/api/auth/register", async (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).json({
      error: "username and password are required",
    });
  }

  const passwordHash = await bcrypt.hash(password, 10);

  const user = {
    id: String(users.length + 1),
    username,
    passwordHash,
  };

  users.push(user);
```

```
  res.status(201).json({
    id: user.id,
    username: user.username,
  });
});

app.post("/api/auth/login", async (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).json({
      error: "username and password are required",
    });
  }

  const user = users.find(u => u.username === username);
  if (!user) {
    return res.status(401).json({
      error: "Invalid credentials",
    });
  }

  const isValid = await bcrypt.compare(password, user.passwordHash);
  if (!isValid) {
    return res.status(401).json({
      error: "Invalid credentials",
    });
  }

  // Создание access-токена
  const accessToken = jwt.sign(
    {
      sub: user.id,
      username: user.username,
    },
    JWT_SECRET,
    {
      expiresIn: ACCESS_EXPIRES_IN,
    }
  );

  res.json({
    accessToken,
  });
});

app.listen(PORT, () => {
```

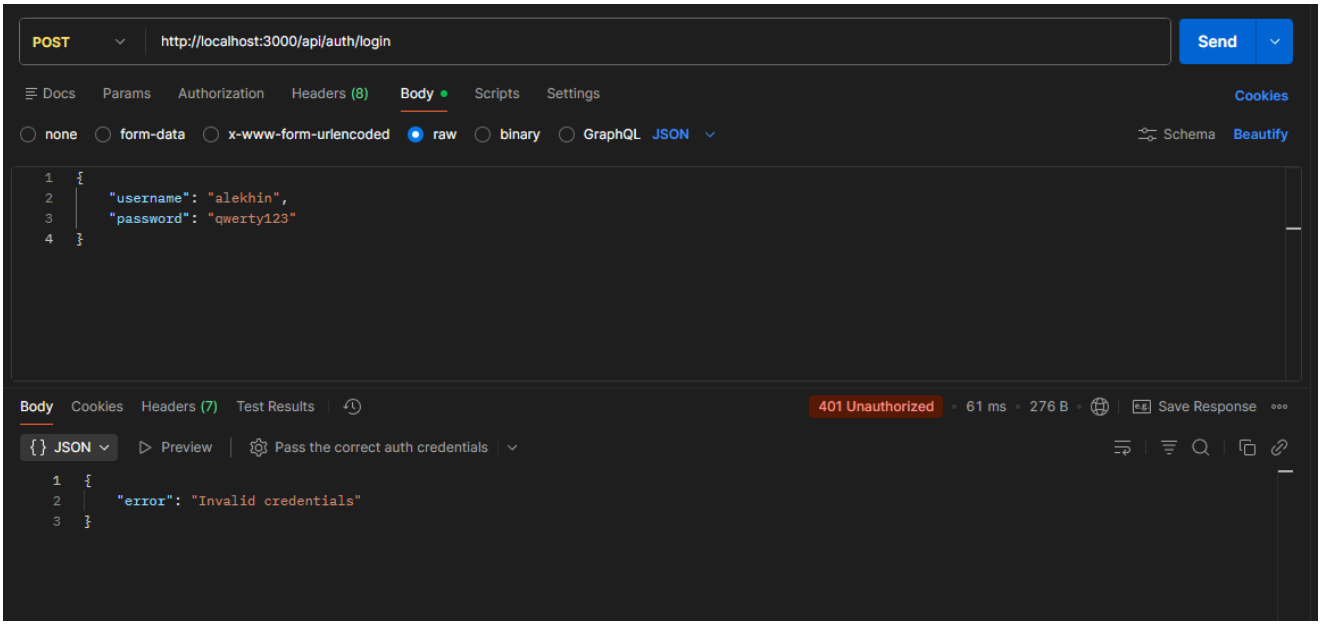
```
  console.log(`Сервер запущен на http://localhost:${PORT}`);
});
```

Так выглядит процесс выдачи токенов. Протестируем в Postman.

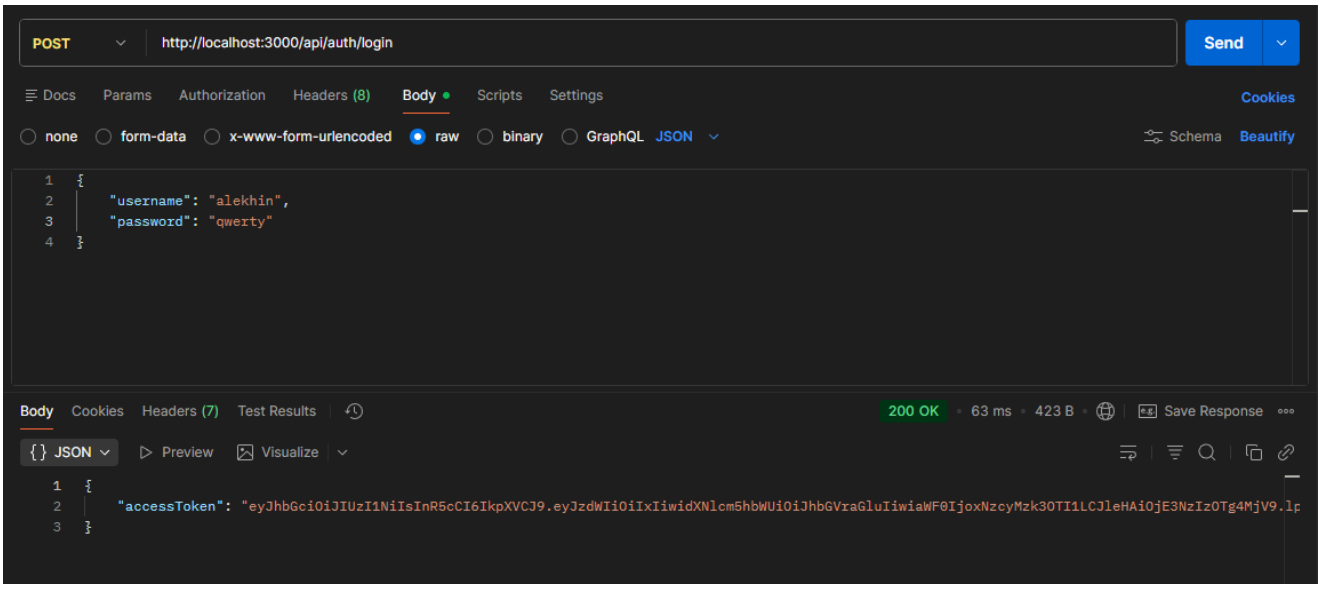
Регистрируем пользователя:



Проверяем вход с неверными данными:



Запрос отклонён. Теперь с верными:



Мы получили строку такого вида:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwidXNlcm5hbWUiOiJhbGVraGl
uIiwiaWF0IjoxNzcyMzk3OTI1LCJleHAiOjE3NzIzOTg4MjV9.lpYoWoIv-
WSornoxfhyfVYLJh40w-l7OPYLKwPS7r1o
```

Это и есть наш токен.

Защищённые маршруты

Для создания защищённых маршрутов (тех, для которых пользователю необходимо быть аутентифицированным - иметь JWT) необходимо создать middlewarre:

```
const jwt = require("jsonwebtoken");

const JWT_SECRET = "access_secret";

function authMiddleware(req, res, next) {
  const header = req.headers.authorization || "";

  // Ожидаем формат: Bearer <token>
  const [scheme, token] = header.split(" ");

  if (scheme !== "Bearer" || !token) {
    return res.status(401).json({
      error: "Missing or invalid Authorization header",
    });
  }

  try {
    const payload = jwt.verify(token, JWT_SECRET);

    // сохраняем данные токена в req
    req.user = payload; // { sub, username, iat, exp }
```

```
    next();
  } catch (err) {
    return res.status(401).json({
      error: "Invalid or expired token",
    });
  }
}
```

Валидируем JWT мы с помощью функции:

```
jwt.verify(token, JWT_SECRET)
```

На выходе получаем полезную нагрузку токена в формате:

```
{ sub, username, iat, exp }
```

Обычно в заголовках JWT передают как `Bearer {token}`, поэтому в middleware именно такой формат мы и будем ожидать.

Теперь создадим маршрут `/api/auth/me`, который будет валидировать токен и возвращать объект текущего пользователя. В нужный маршрут подставим `authMiddleware`
Код будет выглядеть так:

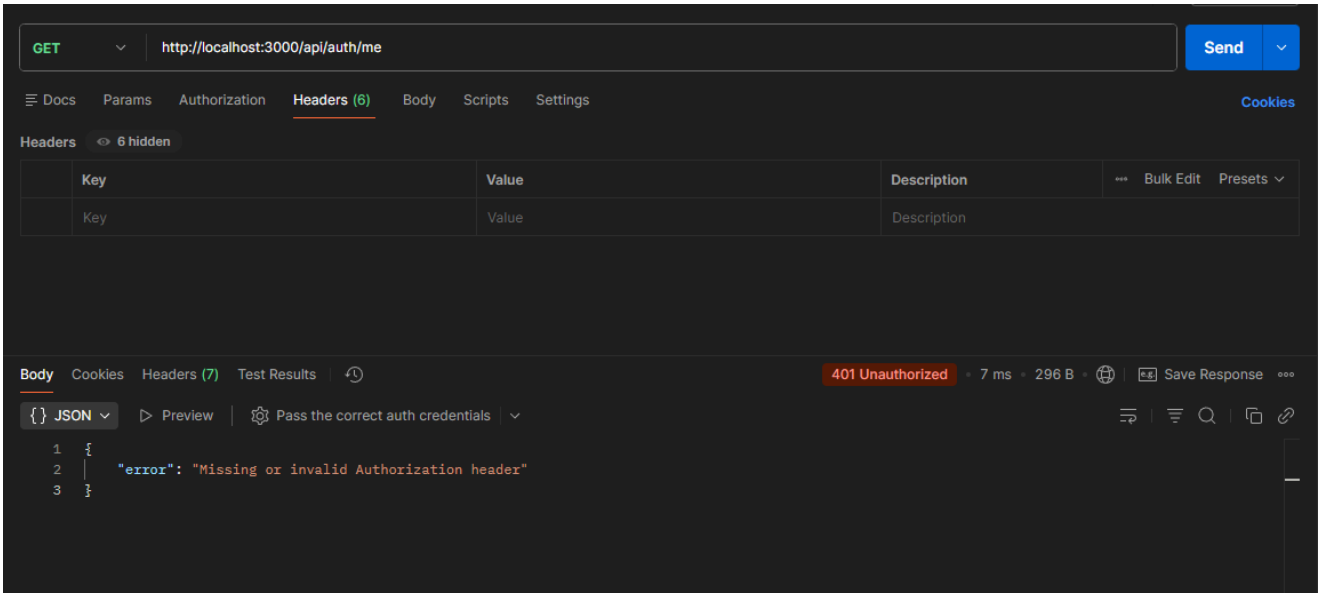
```
const express = require("express");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");

const app = express();
app.use(express.json());

const PORT = 3000;
// Рандомная строка, задающая подпись (гарантия, что токен выдан именно
твоим сервером)
const JWT_SECRET = "access_secret";
// Время жизни токена
const ACCESS_EXPIRES_IN = "15m";

// { id, username, passwordHash }
const users = [];

function authMiddleware(req, res, next) {
  const header = req.headers.authorization || "";

  // Ожидаем формат: Bearer <token>
  const [scheme, token] = header.split(" ");
```

```
  if (scheme !== "Bearer" || !token) {
    return res.status(401).json({
      error: "Missing or invalid Authorization header",
    });
  }

  try {
    const payload = jwt.verify(token, JWT_SECRET);

    // сохраняем данные токена в req
    req.user = payload; // { sub, username, iat, exp }

    next();
  } catch (err) {
    return res.status(401).json({
      error: "Invalid or expired token",
    });
  }
}

app.get("/api/auth/me", authMiddleware, (req, res) => {
  // sub мы положили в токен при login
  const userId = req.user.sub;

  const user = users.find(u => u.id === userId);

  if (!user) {
    return res.status(404).json({
      error: "User not found",
    });
  }

  // никогда не возвращаем passwordHash
  res.json({
    id: user.id,
    username: user.username,
  });
});

app.post("/api/auth/register", async (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).json({
      error: "username and password are required",
    });
  }

  const passwordHash = await bcrypt.hash(password, 10);
```

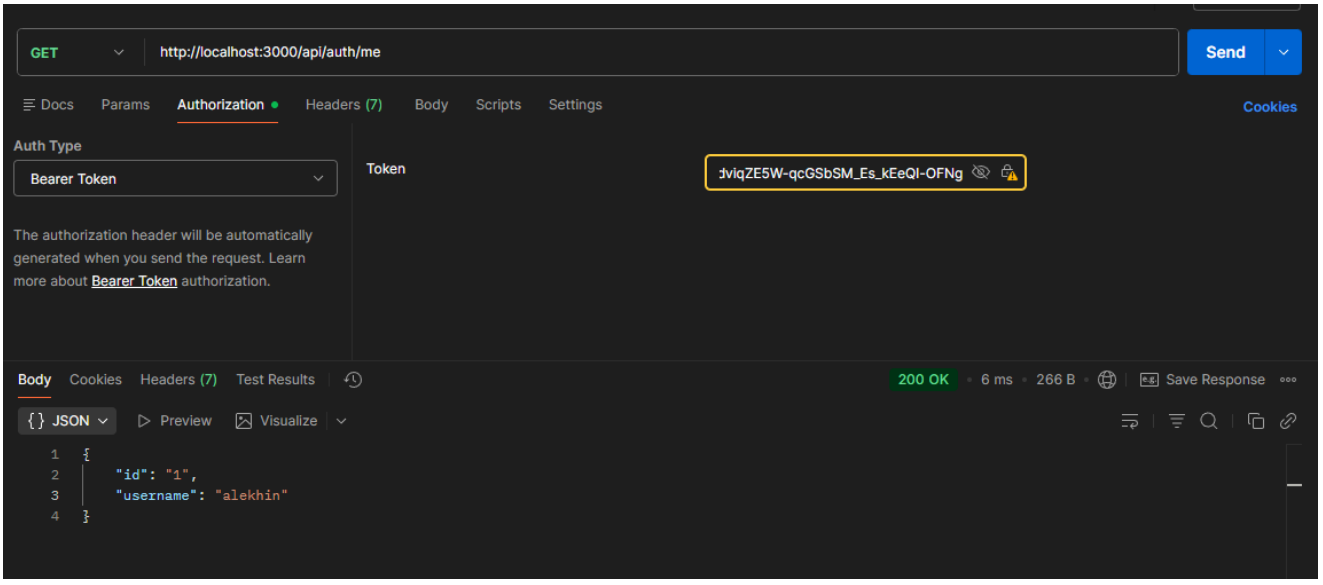
```
  const user = {
    id: String(users.length + 1),
    username,
    passwordHash,
  };

  users.push(user);

  res.status(201).json({
    id: user.id,
    username: user.username,
  });
});

app.post("/api/auth/login", async (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).json({
      error: "username and password are required",
    });
  }

  const user = users.find(u => u.username === username);
  if (!user) {
    return res.status(401).json({
      error: "Invalid credentials",
    });
  }

  const isValid = await bcrypt.compare(password, user.passwordHash);
  if (!isValid) {
    return res.status(401).json({
      error: "Invalid credentials",
    });
  }

  // Создание access-токена
  const accessToken = jwt.sign(
    {
      sub: user.id,
      username: user.username,
    },
    JWT_SECRET,
    {
      expiresIn: ACCESS_EXPIRES_IN,
    }
  );
```

```
  res.json({
    accessToken,
  });
});

app.listen(PORT, () => {
  console.log(`Сервер запущен на http://localhost:${PORT}`);
});
```

Попробуем обратиться к маршруту без токена:



А теперь с токеном:



Теперь мы получили объект "текущего", т.е. авторизованного пользователя.

Практическое задание

Необходимо доработать серверное приложение на node.js из практического занятия №7: реализуйте выдачу токена при входе в систему и добавьте защищённый маршрут `GET /api/auth/me`, который будет возвращать объект текущего пользователя.

Защитите следующие маршруты:

- `GET /api/products/:id`
- `PUT /api/products/:id`
- `DELETE /api/products/:id`

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой. Ссылка подгружается в соответствующий раздел СДО: Задания для самостоятельной работы -> Практические занятия 7-8.